CAPÍTULO 1: INICIO - SOBRE EL CONTROL DE VERSIONES

¿Qué es el control de versiones?

El control de versiones es un sistema que registra los cambios hechos a uno o varios archivos a lo largo del tiempo, de modo que se puedan recuperar versiones anteriores cuando sea necesario. Permite comparar cambios, ver quién modificó qué y volver a un estado anterior si algo sale mal.

Existen tres tipos de sistemas:

Locales: guardan las versiones en una base de datos en el mismo computador. Son simples, pero no permiten colaborar.

Centralizados: un único servidor guarda todas las versiones y los usuarios descargan los archivos desde allí. Facilitan la colaboración, pero si el servidor falla se puede perder el historial.

Distribuidos (como Git): cada usuario tiene una copia completa del repositorio, con todo su historial. Si el servidor falla, cualquier copia puede restaurarlo.

1. Breve historia de Git

Git nació en 2005, cuando la comunidad que desarrollaba el kernel de Linux dejó de usar la herramienta propietaria que empleaba hasta entonces. Linus Torvalds y su comunidad crearon un sistema nuevo con estos objetivos: rapidez, diseño sencillo, soporte para desarrollo no lineal (muchas ramas), carácter totalmente distribuido y capacidad de manejar proyectos muy grandes.

2. ¿Qué es Git?

Git se diferencia de otros sistemas por varias razones:

Instantáneas, no diferencias: Git guarda una “fotografía” del estado de todos los archivos en cada commit, en lugar de guardar solo la lista de cambios.

Operaciones locales: casi todo se hace en tu computador, sin conexión a un servidor, por lo que es muy rápido.

Integridad: todo se verifica mediante una suma de comprobación (hash SHA-1), así que Git detecta cualquier corrupción o cambio no registrado.

Casi siempre solo añade datos: es difícil perder información, porque lo que se confirma queda guardado.

Los tres estados

Un archivo en Git puede estar en uno de tres estados:

1. Modificado (modified): se cambió, pero aún no se ha guardado.
2. Preparado (staged): se marcó para incluirlo en el próximo commit (git add).
3. Confirmado (committed): quedó guardado de forma segura en la base de datos local (git commit).

Esto lleva a las tres secciones de un proyecto Git: el **directorio de trabajo**, el **área de preparación (staging area)** y el **directorio de Git (repositorio)**.

3. La línea de comandos

Existen interfaces gráficas para Git, pero la línea de comandos es la forma más completa de usarlo: todas las funciones están disponibles ahí, y quien domina la línea de comandos puede usar cualquier interfaz gráfica, pero no al revés. En esta actividad se utilizó **Git Bash**.

4. Instalación de Git

En Windows, Git se descarga desde la página oficial (https://git-scm.com) y se instala con un asistente. Incluye Git Bash, la terminal usada para ejecutar los comandos. Para comprobar que quedó instalado se ejecuta:

```
git --version
```

5. Configuración inicial

Antes de usar Git por primera vez se debe indicar el nombre y el correo del usuario, ya que quedan registrados en cada commit:

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu-correo@ejemplo.com"
```

La opción --global hace que esta configuración se aplique a todos los repositorios del computador. Para revisar la configuración se usa:

```
git config --list
```

6. ¿Cómo obtener ayuda?

Git incluye ayuda integrada. Cualquiera de estas formas muestra el manual de un comando:

```
git help <comando>
git <comando> --help
```

Por ejemplo: git help config.

Resumen

Git es un sistema de control de versiones distribuido, rápido y confiable. Guarda instantáneas del proyecto, permite trabajar sin conexión y maneja tres estados de los archivos (modificado, preparado y confirmado). Tras instalarlo y configurar el nombre y el correo, ya se puede empezar a trabajar con repositorios.

Referencias

Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. https://git-scm.com/book/es/v2